

Reviewer Pack — Minimal Rank of Noncommutative Polynomial Representations an...

Entry d7c8c75e5a56 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 11:28:25 UTC

Minimal Rank of Noncommutative Polynomial Representations and SAT Clause Complexity

Entry ID: d7c8c75e5a56 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 11:28:25 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra **Field B** (complexity object): SAT (Satisfiability) Clause Complexity

Statement:

For every CNF φ with n variables, the minimal rank of a noncommutative polynomial representation $R(\varphi)$ is linearly correlated with its clause complexity $c(\varphi)$, such that $\text{rank}(R(\varphi)) = \Theta(c(\varphi))$.

Rationale (proposer's reasoning):

Noncommutative algebra provides an alternative framework for studying polynomials that may reveal inherent structures in SAT clauses, potentially leading to new insights into the hardness of satisfiability problems.

Taxonomy category: noncommutative_algebra_satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a953eb7b1db30889

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal rank of noncommutative polynomial representations and clause complexity is ≥ 0.8 for at least 80% of the seeds, with no seed's correlation coefficient < 0.6 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative polynomial representation" AND "SAT clause complexity" - "minimal rank noncommutative algebra" AND "satisfiability complexity" - "algebraic representation" AND "SAT problem"

Top relevant hits considered: - [<http://arxiv.org/abs/2505.18460v1>] Mapping between Spin-Glass Three-Dimensional (3D) Ising Model and Boolean Satisfiability Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
                clause[1] *= -1
            clauses.append(clause)
        return clauses

    def compute_clause_complexity(cnf):
        return len(cnf)

    def construct_noncommutative_polynomial_representation(cnf):
        n = max(abs(x) for clause in cnf for x in clause)
        R = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in cnf:
            if len(clause) == 2:
                i, j = abs(clause[0]), abs(clause[1])
                sign_i, sign_j = clause[0] // i, clause[1] // j
                R[i][j] += sign_i * sign_j
        return R

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
```

```

rank = 0
for col in range(n):
    pivot_row = -1
    for row in range(m):
        if matrix[row][col] != 0:
            pivot_row = row
            break
    if pivot_row == -1:
        continue
    rank += 1
    for i in range(m):
        if i != pivot_row and matrix[i][col] != 0:
            factor = matrix[i][col] / matrix[pivot_row][col]
            for j in range(n):
                matrix[i][j] -= factor * matrix[pivot_row][j]

return rank

n_values = [5, 10, 15, 20, 30, 40]
ranks = []
complexities = []

for n in n_values:
    cnf = generate_cnf(n)
    complexity = compute_clause_complexity(cnf)
    R = construct_noncommutative_polynomial_representation(cnf)
    rank = matrix_rank(R)

    ranks.append(rank)
    complexities.append(complexity)

correlation_coefficient = sum((ranks[i] - mean_ranks) * (complexities[i] - mean_complexities))
mean_ranks = sum(ranks) / len(ranks)
mean_complexities = sum(complexities) / len(complexities)

return {
    "metric_name": "Pearson's correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": 0.6 <= correlation_coefficient < 0.8,
    "counterexample": "" if 0.6 <= correlation_coefficient < 0.8 else f"Correlation coefficient is {correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```
if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std=0.0000 support_fraction=1.0000")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std=0.0000 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient out of range\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

- -STDERR- -

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b7fc82e.py", line 94, in <module>
    result = run_trial(seed)
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lb7fc82e.py", line 75, in run_trial
    correlation_coefficient = sum((ranks[i] - mean_ranks) * (complexities[i] -
mean_complexities) for i in range(len(n_values))) / len(n_values)
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lb7fc82e.py", line 75, in <genexpr>
    correlation_coefficient = sum((ranks[i] - mean_ranks) * (complexities[i] -
mean_complexities) for i in range(len(n_values))) / len(n_values)
                                ^^^^^^^^^^^^^
```

NameError: cannot access free variable 'mean_ranks' where it is not associated with a value in enclosing scope

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and meeting the pre-registered support condition. | next: Review the code for potential bugs that could cause a crash during execution and ensure that all necessary variables are defined within their appropriate scope.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23960
2	preregistration	ollama_remote	glm4:latest	0	0	9056
3	novelty	ollama_remote	glm4:latest	0	0	12939

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	17935
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18568
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15180
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17182
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12567
9	judge	ollama_remote	glm4:latest	0	0	8853

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 136241 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/d7c8c75e5a56.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/d7c8c75e5a56.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/d7c8c75e5a56.tar.gz* (if generated)